

Forward and Inverse Kinematics of the `new_arm` Manipulator

Robot description: `new_arm_description`

Solver package: `new_arm_ik`

2026-05-31

Abstract

This document describes the kinematics of the `new_arm`, a 4-DOF revolute manipulator, as implemented in the `new_arm_ik` ROS 2 package. The forward kinematics is built directly from the robot's URDF so that the solver and the model rendered in RViz are identical by construction. The inverse kinematics is formulated as a bounded nonlinear least-squares problem with a continuity-preserving regularizer and a random-restart fallback. We also document a tuning fix that reduced the median position residual from 20 mm to below 1 μm on reachable targets.

Contents

1	Robot model	1
2	Rotation primitives	2
2.1	Roll-pitch-yaw to a rotation matrix	2
2.2	Axis-angle (Rodrigues) rotation	2
2.3	Homogeneous transforms	3
3	Forward kinematics	3
4	Inverse kinematics	3
4.1	Problem statement	3
4.2	Residual and cost	3
4.3	The bias-accuracy trade-off in λ	4
4.4	Random-restart fallback	4
5	Solver parameters	4
6	The tuning fix	4
7	ROS 2 integration	5

1 Robot model

The `new_arm` is a serial chain of four revolute joints connecting five links:

$$\text{arm_base} \xrightarrow{q_1} \text{after_base_full} \xrightarrow{q_2} \text{link1} \xrightarrow{q_3} \text{link2} \xrightarrow{q_4} \text{end_effector}.$$

The kinematic parameters are read verbatim from `new_arm_description/urdf/new_arm.urdf`. Each joint i contributes a fixed origin transform (a translation $\mathbf{p}_i \in \mathbb{R}^3$ and a fixed orientation given by roll-pitch-yaw angles $\mathbf{r}_i = (\phi_i, \theta_i, \psi_i)$) followed by a rotation of q_i about the joint axis $\hat{\mathbf{a}}_i \in \mathbb{R}^3$. Table 1 lists the values.

Table 1: Joint parameters from the URDF. Origins $\mathbf{p}_i$ are in metres; all fixed RPY orientations are zero. Limits are symmetric.

Joint	Axis $\hat{\mathbf{a}}_i$	Origin $\mathbf{p}_i = (x, y, z)$ m	Limit rad	Effort
<code>after_base_full_joint</code>	(0, 0, 1)	(0.00043, -0.00184, 0.08396)	$[-3.14, 3.14]$	10.0
<code>link1_joint</code>	(0, 1, 0)	(0.00814, -0.00043, 0.00162)	$[-3.14, 3.14]$	10.0
<code>link2_joint</code>	(0, 1, 0)	(0.00100, -0.00856, 0.12840)	$[-3.14, 3.14]$	10.0
<code>end_effector_joint</code>	(0, 1, 0)	(0.12532, 0.01099, -0.05778)	$[-3.14, 3.14]$	5.0

The first joint rotates about z (a base yaw); the remaining three rotate about y (pitches). The reachable workspace is a region of radius ≈ 0.25 m. In the current interface the fourth joint q_4 (`end_effector_joint`) is held at 0; the position IK has three Cartesian targets and four joint variables, so the arm is kinematically redundant by one degree of freedom.

2 Rotation primitives

2.1 Roll–pitch–yaw to a rotation matrix

URDF `origin` orientations use the ROS fixed-axis roll–pitch–yaw convention. With roll ϕ about x , pitch θ about y , and yaw ψ about z , the composed rotation is the intrinsic $z y x$ product

$$R(\phi, \theta, \psi) = R_z(\psi) R_y(\theta) R_x(\phi), \quad (1)$$

where

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}, \quad R_z(\psi) = \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

with the shorthand $c_\alpha = \cos \alpha$, $s_\alpha = \sin \alpha$. Expanding the product gives the closed form implemented in `rpj_to_matrix`:

$$R(\phi, \theta, \psi) = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}. \quad (2)$$

2.2 Axis–angle (Rodrigues) rotation

Each actuated joint rotates by q_i about a unit axis $\hat{\mathbf{a}} = (a_x, a_y, a_z)$. The rotation matrix is given by Rodrigues’ formula

$$R(\hat{\mathbf{a}}, q) = I + (\sin q) [\hat{\mathbf{a}}]_\times + (1 - \cos q) [\hat{\mathbf{a}}]_\times^2, \quad (3)$$

where $[\hat{\mathbf{a}}]_\times$ is the skew-symmetric cross-product matrix

$$[\hat{\mathbf{a}}]_\times = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}.$$

Written componentwise (the form in `axis_angle_matrix`), with $c = \cos q$, $s = \sin q$, and $C = 1 - c$:

$$R(\hat{\mathbf{a}}, q) = \begin{bmatrix} c + a_x^2 C & a_x a_y C - a_z s & a_x a_z C + a_y s \\ a_y a_x C + a_z s & c + a_y^2 C & a_y a_z C - a_x s \\ a_z a_x C - a_y s & a_z a_y C + a_x s & c + a_z^2 C \end{bmatrix}. \quad (4)$$

2.3 Homogeneous transforms

A rotation $R \in \text{SO}(3)$ and translation $\mathbf{t} \in \mathbb{R}^3$ are packed into a homogeneous transform $T \in \text{SE}(3)$:

$$T(R, \mathbf{t}) = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}. \quad (5)$$

3 Forward kinematics

The pose of the end-effector relative to the base is the ordered product of, for each joint, the fixed origin transform followed by the variable joint rotation:

$$T_{\text{ee}}^{\text{base}}(\mathbf{q}) = \prod_{i=1}^4 \underbrace{T(R(\mathbf{r}_i), \mathbf{p}_i)}_{\text{fixed origin}} \underbrace{T(R(\hat{\mathbf{a}}_i, q_i), \mathbf{0})}_{\text{joint } q_i} \quad (6)$$

where $\mathbf{q} = (q_1, q_2, q_3, q_4)$. This is exactly the loop in `Chain.fk`. The end-effector position used by the IK is the translation block

$$\mathbf{x}(\mathbf{q}) = [T_{\text{ee}}^{\text{base}}(\mathbf{q})]_{1:3,4} \in \mathbb{R}^3. \quad (7)$$

Because the parameters $\{\mathbf{p}_i, \mathbf{r}_i, \hat{\mathbf{a}}_i\}$ are parsed straight from the URDF, $\mathbf{x}(\mathbf{q})$ coincides with the frame that `robot_state_publisher` broadcasts to RViz; there is no separate hand-derived model to drift out of sync.

As a reference, the home pose $\mathbf{q} = \mathbf{0}$ gives

$$\mathbf{x}(\mathbf{0}) \approx (0.1349, 0.0002, 0.1562) \text{ m}.$$

4 Inverse kinematics

4.1 Problem statement

Given a Cartesian target $\mathbf{x}^* \in \mathbb{R}^3$, find joint angles $\mathbf{q} \in \mathbb{R}^4$ within the box limits $[\mathbf{q}_{\min}, \mathbf{q}_{\max}]$ whose forward kinematics reaches the target. Since the map $\mathbf{x}(\mathbf{q})$ is nonlinear and the arm is redundant, we solve a regularized nonlinear least-squares problem rather than a closed-form inverse.

4.2 Residual and cost

Let $\mathbf{q}_{\text{ref}}$ be a reference configuration (the current pose, for continuity). Define the stacked residual $\mathbf{r} : \mathbb{R}^4 \rightarrow \mathbb{R}^7$

$$\mathbf{r}(\mathbf{q}) = \begin{bmatrix} w_p (\mathbf{x}(\mathbf{q}) - \mathbf{x}^*) \\ \lambda (\mathbf{q} - \mathbf{q}_{\text{ref}}) \end{bmatrix} \in \mathbb{R}^{3+4}, \quad (8)$$

combining a *position* term (weight w_p , three rows) and a *regularization* term (weight λ , four rows). The solver minimizes

$$\mathbf{q}^* = \arg \min_{\mathbf{q}_{\min} \leq \mathbf{q} \leq \mathbf{q}_{\max}} \frac{1}{2} \|\mathbf{r}(\mathbf{q})\|_2^2 = \arg \min_{\mathbf{q}} \frac{1}{2} \left(w_p^2 \|\mathbf{x}(\mathbf{q}) - \mathbf{x}^*\|^2 + \lambda^2 \|\mathbf{q} - \mathbf{q}_{\text{ref}}\|^2 \right). \quad (9)$$

The position term drives the end-effector to the target. The regularization term has two jobs: it resolves the one-dimensional redundancy by selecting, among all exact solutions, the one nearest $\mathbf{q}_{\text{ref}}$, and it keeps successive solves close together so the arm moves smoothly in RViz with no joint flips.

This is solved with `scipy.optimize.least_squares` (Trust Region Reflective), which respects the box bounds and uses a finite-difference Jacobian $J = \partial \mathbf{r} / \partial \mathbf{q}$.

4.3 The bias–accuracy trade-off in λ

At a solution the two objectives compete: increasing λ pulls $\mathbf{q}^\star$ toward $\mathbf{q}_{\text{ref}}$ at the expense of position accuracy. Concretely, near a minimum the position error scales like

$$\|\mathbf{x}(\mathbf{q}^\star) - \mathbf{x}^\star\| \sim \frac{\lambda^2}{w_p^2} \|J_x^+\| \|\mathbf{q}^\star - \mathbf{q}_{\text{ref}}\|, \quad (10)$$

where J_x is the 3×4 position Jacobian and J_x^+ its pseudoinverse. The residual therefore grows roughly quadratically in λ . This is the mechanism behind the bug described in Section 6: an over-large λ produced centimetre-scale errors on every target.

4.4 Random-restart fallback

Even with $\lambda \rightarrow 0$, a single local optimization seeded from one point can stall in a local minimum (the position objective is non-convex), which empirically happened on about 2% of reachable targets, with residuals up to ~ 100 mm. The solver therefore uses a two-phase strategy:

1. **Continuity solve.** Seed at $\mathbf{q}_{\text{init}} = \mathbf{q}_{\text{ref}}$ = the current pose and solve once. If the resulting residual is within a tolerance ε (default 0.5 mm), accept it. This is the fast common path and yields smooth motion.
2. **Restart fallback.** Otherwise, draw up to N random seeds uniformly from the joint box, $\mathbf{q}^{(k)} \sim \mathcal{U}(\mathbf{q}_{\text{min}}, \mathbf{q}_{\text{max}})$, solve from each (regularizing toward that seed, i.e. no continuity bias), and keep the lowest-residual result. Stop early as soon as the tolerance is met.

Formally, the returned solution is

$$\mathbf{q}^\star = \arg \min_{\mathbf{q} \in \{\mathbf{q}^{(0)}, \dots, \mathbf{q}^{(N)}\}} \|\mathbf{x}(\mathbf{q}) - \mathbf{x}^\star\|, \quad (11)$$

where $\mathbf{q}^{(0)}$ is the continuity solve and $\mathbf{q}^{(1)}, \dots, \mathbf{q}^{(N)}$ are the restart solves (evaluated only when needed). The hard targets pay the extra cost; the easy ones do not.

5 Solver parameters

Table 2: IKSolver parameters and their meaning.

Symbol	Code name	Default
w_p	<code>position_weight</code>	1.0
λ	<code>regularization</code>	10^{-4}
ε	<code>tolerance</code>	5×10^{-4} m
N	<code>restarts</code>	6
–	<code>max_iter</code>	200

6 The tuning fix

The original solver shipped with $\lambda = 0.05$ and no restart fallback. By the relation in the trade-off subsection, that regularization weight was large enough to dominate the position objective, biasing every solution toward the seed. The effect was measured by generating guaranteed-reachable targets (sample random $\mathbf{q}_{\text{true}}$, compute $\mathbf{x}(\mathbf{q}_{\text{true}})$ by FK, then ask the IK to recover the target); this isolates solver quality from workspace limits.

Table 3: Position residual on reachable targets, before and after. Lower is better; “miss rate” is the fraction of targets with residual > 1 mm.

Configuration	Median	Worst case	Miss rate
$\lambda = 0.05$ (original)	20 mm	46 mm	100 %
$\lambda = 0.005$	0.36 mm	3.1 mm	12 %
$\lambda = 0$, single start	0 mm	108 mm	2 %
$\lambda = 10^{-4}$ + restart (current)	$< 1 \mu\text{m}$	0.022 mm	0 %

The adopted configuration ($\lambda = 10^{-4}$ with the two-phase restart fallback) achieves sub-micrometre median accuracy and zero misses over 500 reachable targets, while preserving smooth motion: stepping the target along a line produced a maximum single-step joint change of only 0.68° .

7 ROS 2 integration

The solver is wrapped by `ik_gui_node.py`, a node that:

- parses the chain once with `parse_urdf_chain(urdf, root, tip)`;
- exposes a Tkinter panel for entering an (x, y, z) target in metres;
- calls `solver.solve(target, q_init=self.current_q)` on demand, feeding the current pose as the continuity reference;
- publishes the resulting joint vector on `/joint_states` at 20 Hz, which `robot_state_publisher` turns into TF frames for RViz.

No change to the node was required for the tuning fix; it benefits automatically because the solver’s public `solve` signature is unchanged.

Running it.

```
colcon build --packages-select new_arm_description new_arm_ik
source install/setup.bash
ros2 launch new_arm_ik ik_interface.launch.py
```